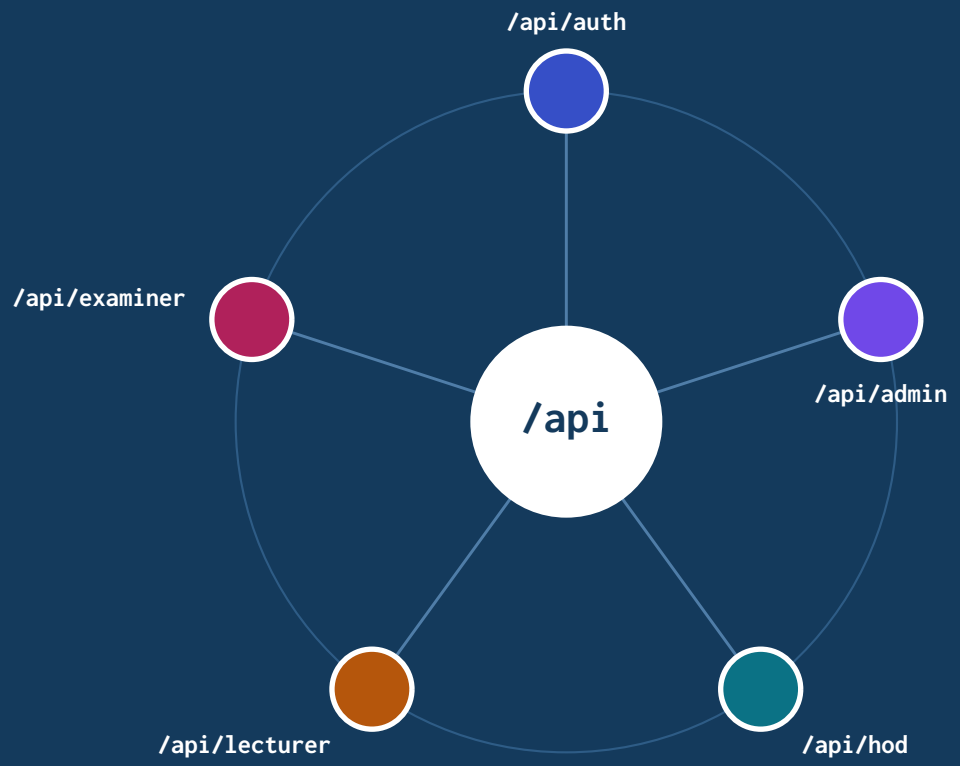
API REFERENCE

API Map

Student Exam & Results Management System

Wayamba University of Sri Lanka

27	15	5	3
operations	route files	API groups	database tables

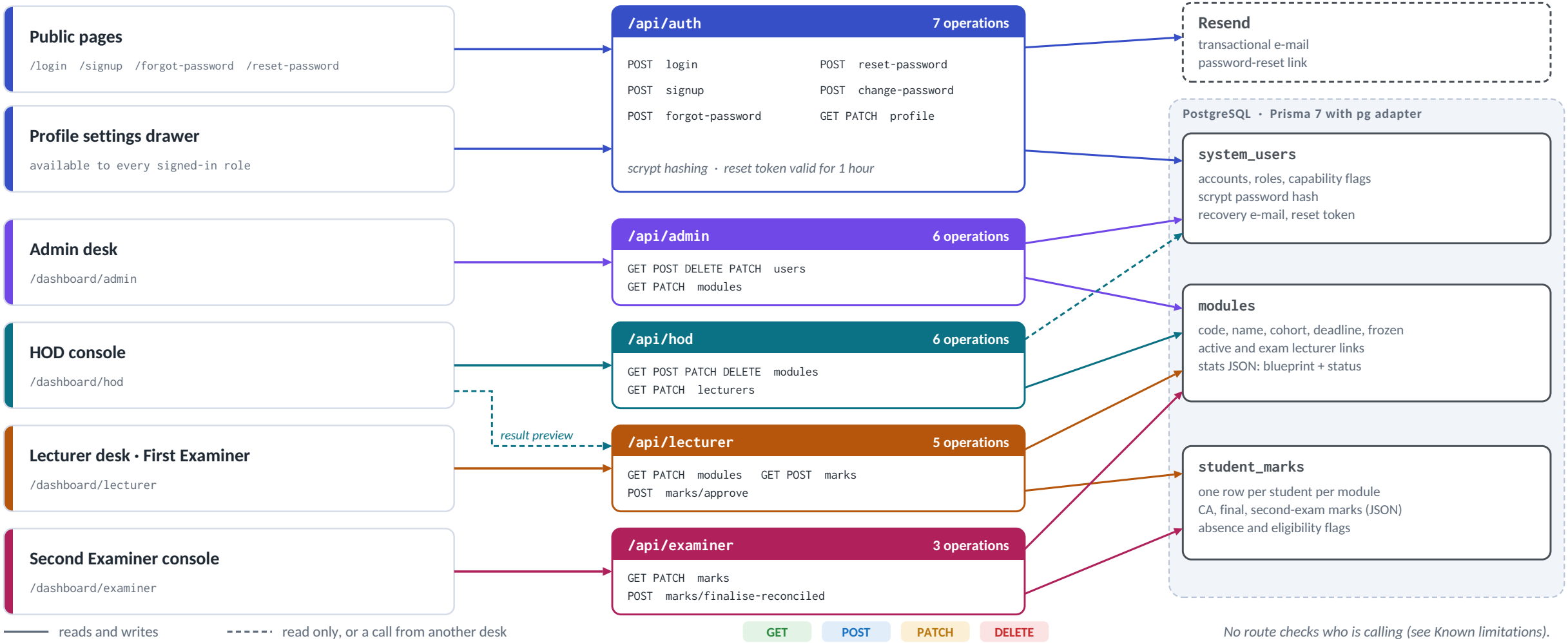


Source: dasikav-commits/Student-Exam-Results-Management · code as of commit f49d5c9 (Dasika branch), 19 October 2026

OVERVIEW

Architecture map

Who calls which API group, and which tables and services each group touches



Endpoint index

Every operation, who calls it from the UI (1 of 2)

Method	Endpoint	What it does	Used by
Authentication and accounts `/api/auth`			
POST	**`/api/auth/login`**	Sign in with e-mail and password	Login page
POST	**`/api/auth/signup`**	Self-service lecturer sign-up (university e-mail only)	Sign-up page
POST	**`/api/auth/forgot-password`**	Request a password-reset link by e-mail	Forgot-password page
POST	**`/api/auth/reset-password`**	Set a new password using the reset token	Reset-password page
POST	**`/api/auth/change-password`**	Change password (current password required)	Profile drawer
GET	**`/api/auth/profile`**	Read name and recovery e-mail	Profile drawer
PATCH	**`/api/auth/profile`**	Update name and recovery e-mail	Profile drawer
System administration `/api/admin`			
GET	**`/api/admin/users`**	List all user accounts	Admin desk
POST	**`/api/admin/users`**	Create a user account	Admin desk
DELETE	**`/api/admin/users`**	Delete a user account	Admin desk
PATCH	**`/api/admin/users`**	Switch one capability flag on or off	Admin desk
GET	**`/api/admin/modules`**	List modules with Second Examiner assignment state	Admin desk
PATCH	**`/api/admin/modules`**	Assign or clear a module's Second Examiner	Admin desk

Endpoint index

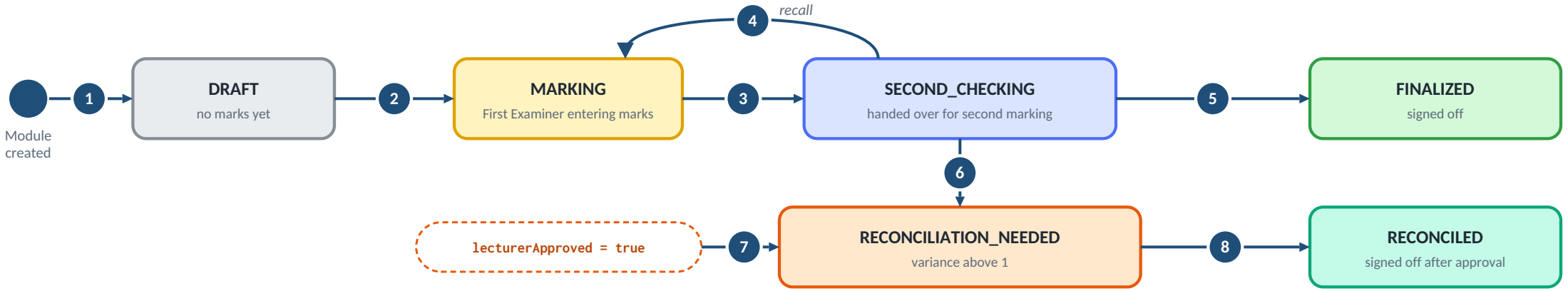
Every operation, who calls it from the UI (2 of 2)

Method	Endpoint	What it does	Used by
HOD console `/api/hod`			
GET	**`/api/hod/modules`**	List modules with their lecturers and blueprint	HOD console
POST	**`/api/hod/modules`**	Create a module and assign its First Examiner	HOD console
PATCH	**`/api/hod/modules`**	Edit a module, freeze or unfreeze it, or re-assign lecturers	HOD console
DELETE	**`/api/hod/modules`**	Delete a module and all its marks	HOD console
GET	**`/api/hod/lecturers`**	Lecturer roster with module workloads	HOD console
PATCH	**`/api/hod/lecturers`**	Replace one lecturer's module assignments	HOD console
Lecturer desk (First Examiner) `/api/lecturer`			
GET	**`/api/lecturer/modules`**	Modules where the user is First or Second Examiner	Lecturer desk
PATCH	**`/api/lecturer/modules`**	Submit or recall a marksheet, save the blueprint, set cohort size	Lecturer desk
GET	**`/api/lecturer/marks`**	Load a module's marksheet (all students)	Lecturer desk, HOD console
POST	**`/api/lecturer/marks`**	Save the First Examiner's marks and roster changes	Lecturer desk
POST	**`/api/lecturer/marks/approve`**	Approve the result after a variance	Lecturer desk
Second Examiner console `/api/examiner`			
GET	**`/api/examiner/marks`**	List assigned modules, or load the second-marking ledger	Examiner console
PATCH	**`/api/examiner/marks`**	Save second marks and optionally finalise	Examiner console
POST	**`/api/examiner/marks/finalise-reconciled`**	Close a marksheet after reconciliation	Examiner console

WORKFLOW

Marksheet lifecycle

Status values of Module.stats.marksheetStatus and the call that moves each one



First Examiner can edit marks, blueprint and cohort
DRAFT and MARKING

From submission on, First Examiner edits return 409
FINALIZED, RECONCILIATION_NEEDED and RECONCILED cannot be re-opened from the Lecturer Desk

HOD freeze (isFrozen) blocks First Examiner edits and transitions with 403, and second marking and Second Examiner assignment with 409, in every state

#	Transition and call	Guard and effect
1	**start to DRAFT** `POST /api/hod/modules`	Module is created with no CA components. The HOD sets cohort size and deadline.
2	**DRAFT to MARKING** `POST /api/lecturer/marks`	The first successful save advances the status automatically and records `marksheetStartedAt`.

3	**MARKING to SECOND_CHECKING**
	`PATCH /api/lecturer/modules`
	`{ marksheetStatus: "SECOND_CHECKING", submittedTo }`
		Submit. Needs a defined blueprint and a complete marksheet (any N answers per section). Recipient FACULTY, HOD or SECOND_EXAMINER; the last needs an assignment.

4	**SECOND_CHECKING to MARKING**
	`PATCH /api/lecturer/modules`
	`{ marksheetStatus: "MARKING" }`
		Recall. Reopens the First Examiner ledger and adds 1 to `recallCount`.

#	Transition and call	Guard and effect

5	**SECOND_CHECKING to FINALIZED**
	`PATCH /api/examiner/marks`
	`{ finalize: true }`
		Every section complete for each eligible student and weighted variance of 1 or less.

6	**SECOND_CHECKING to RECONCILIATION_NEEDED** same call as 5	Weighted variance above 1. Sets `reconciliationRequestedAt` and resets `lecturerApproved` to false.
7	**approval (status unchanged)** `POST /api/lecturer/marks/approve`	First Examiner approves. Sets `lecturerApproved` and `lecturerApprovedAt`.
8	**RECONCILIATION_NEEDED to RECONCILED** `POST /api/examiner/marks/finalise-reconciled`	Needs `lecturerApproved`. Sets `reconciledAt`.

REFERENCE

Authentication and accounts

/api/auth · 7 operations

Method	Endpoint	Request	Success response	Rules and errors
POST	**/api/auth/login** Sign in	`{ email, password }`	`200 { user: { id, email, fullName, role, capabilities: { isHOD, isActiveLec, isExamLec } } }`	400 missing field. 401 unknown e-mail or wrong password. Password checked against a scrypt hash (legacy plain-text still accepted). **No token or cookie is issued.**
POST	**/api/auth/signup** Self-service lecturer sign-up	`{ fullName, email, password }`	`201 { success, user: { id, email, fullName, role } }`	400 validation (name 2-100 characters, e-mail domain must be wayamba.ac.lk or wyb.ac.lk, password 8-50). 409 e-mail already registered. Created as LECTURER with no capabilities; the Admin grants them.
POST	**/api/auth/forgot-password** Request a reset link	`{ email }`	`200 { success: true }`	Always 200, even for unknown e-mails. 400 e-mail missing. Stores a 32-byte token valid for 1 hour and e-mails the link to the account's **recovery e-mail** through Resend (skipped silently if none is saved). 500 if sending fails.
POST	**/api/auth/reset-password** Choose a new password	`{ token, newPassword }`	`200 { success }`	400 missing field, password under 8 characters, or invalid / expired token. Token is cleared after use; new password stored as a scrypt hash.
POST	**/api/auth/change-password** Change password while signed in	`{ email, currentPassword, newPassword }`	`200 { success }`	400 missing field or under 8 characters. 404 unknown user. 403 wrong current password.
GET	**/api/auth/profile** Read profile	query `email`	`200 { fullName, recoveryEmail }`	400 e-mail missing. 404 unknown user.
PATCH	**/api/auth/profile** Update name or recovery e-mail	`{ email, fullName?, recoveryEmail? }`	`200 { fullName, recoveryEmail }`	400 e-mail missing. An empty recoveryEmail clears it. 500 returns the internal error text as `detail`.

REFERENCE

System administration

/api/admin · 6 operations · Admin desk

Method	Endpoint	Request	Success response	Rules and errors
GET	**/api/admin/users** List all accounts	none	`200 [ { id, email, fullName, role, isHod, isActiveLec, isExamLec, createdAt } ]`	Newest first. 500 on failure.
POST	**/api/admin/users** Create an account	`{ email, fullName, role, password, isHod?, isActiveLec?, isExamLec? }`	`201 { success, user: { id, email, fullName, role } }`	400 missing field. 409 e-mail already registered. Password is hashed. The role is upper-cased but **not checked against a list**.
DELETE	**/api/admin/users** Delete an account	query `id`	`200 { success }`	400 id missing. 404 not found. Modules that referenced the user are kept; their lecturer link becomes null.
PATCH	**/api/admin/users** Switch one capability on or off	`{ id, flag, value }` with flag `isHod`, `isActiveLec` or `isExamLec`, value boolean	`200 { success, user: { id, isHod, isActiveLec, isExamLec } }`	400 missing or invalid field or flag. 404 not found.
GET	**/api/admin/modules** Modules with assignment state	none	`200 [ { id, code, name, activeLecturerId, examLecturerId, assignedLecturerName, totalStudents, assignmentLocked } ]`	`assignmentLocked` is true when the module is frozen, closed, or in second checking. `totalStudents` counts student_marks rows.
PATCH	**/api/admin/modules** Assign or clear the Second Examiner	`{ moduleCode, examLecturerId }` where examLecturerId is a user id or null	`200 { success, examLecturerId }`	400 invalid input, same person as the First Examiner, or target lacks Second Examiner capability. 404 module. 409 frozen, submitted or closed: recall the marksheet first.

REFERENCE

HOD console

/api/hod · 6 operations

Method	Endpoint	Request	Success response	Rules and errors
GET	**/api/hod/modules** Modules with lecturers	none	`200 [ { id, code, name, credits, eligibleStudents, deadline, isFrozen, stats, assignedActiveLec, assignedExamLec } ]`	`stats` is returned normalised (final blueprint and exam template rebuilt). Sorted by code.
POST	**/api/hod/modules** Create a module	`{ name, code, lecturerId, eligibleStudents, deadline, components[] }`	`201 { success, module }`	400 invalid field (components must be `CONTINUOUS_ASSESSMENT`, `PRACTICAL` or `LAB_SESSIONS`). 404 lecturer not found. 409 code exists. Starts as DRAFT with no CA components.
PATCH	**/api/hod/modules** Edit, freeze or re-assign (mode picked by the body)	**a** `{ code, isFrozen }` **b** `{ code, name?, eligibleStudents?, deadline?, components?, activeLecturerId? }` **c** `{ code, activeLecturerId?, examLecturerId? }`	`200 { success, id, code }` (mode a also returns `isFrozen`)	400 invalid field. 404 module not found. 409 duplicate code. **No marksheet-status checks.** Mode c does not validate the lecturer ids.
DELETE	**/api/hod/modules** Delete a module	query code	`200 { success, code }`	400 code missing. 404 not found. **Cascades: every student mark of the module is deleted.** No freeze or status check.
GET	**/api/hod/lecturers** Lecturer roster	none	`200 [ { id, fullName, email, isHod, isActiveLec, isExamLec, activeModules[], examModules[] } ]`	LECTURER accounts only, sorted by name.
PATCH	**/api/hod/lecturers** Replace a lecturer's assignments	`{ lecturerId, activeModuleCodes[], examModuleCodes[] }`	`200 { success }`	400 invalid lecturer id. One transaction: clears the lecturer's old links, then sets the new ones. No lock or capability checks.

REFERENCE

Lecturer desk: modules and workflow

/api/lecturer/modules · First Examiner

Method	Endpoint	Request	Success response	Rules and errors
GET	**/api/lecturer/modules** Modules for the signed-in user	query `email` (or `lecturerId`)	`200 [ module + stats + roleInModule + isExaminerViewOnly ]`	`roleInModule` is LECTURER for the First Examiner and EXAMINER for the Second Examiner. Returns `[]` for a missing or invalid id and for `admin-root`. Stats are returned normalised.
PATCH	**/api/lecturer/modules** **Mode A, status:** submit for second checking, or recall	`{ moduleCode, marksheetStatus, submittedTo? }` with submittedTo `FACULTY`, `HOD` or `SECOND_EXAMINER`	`200 { success, marksheetStatus, stats }` or `{ success, unchanged: true }`	400 unknown status, bad recipient, no Second Examiner assigned, blueprint missing, or marksheet incomplete (returns `issues[]`). 403 frozen. 409 closed marksheet or move not allowed. Allowed: DRAFT or MARKING -> SECOND_CHECKING (submit); DRAFT or SECOND_CHECKING -> MARKING (recall).
PATCH	**/api/lecturer/modules** **Mode B, blueprint:** save CA components and final paper	`{ moduleCode, caComponents[], finalBlueprint, secondChecking? }` with secondChecking `required` or `optional`	`200 { success, stats, orphanedMarksCleared? }`	400 structure errors: weights must total exactly 100%; Group A needs total marks; Group B needs 1-50 questions, weights and a valid required-answer count. 403 frozen. 409 submitted or closed. Rebuilds `examTemplate` and removes marks of deleted CA components.
PATCH	**/api/lecturer/modules** **Mode C, cohort:** set the authorised cohort size	`{ moduleCode, eligibleStudents }`	`200 { success, eligibleStudents }`	400 not a whole number from 0 up. 403 frozen. 409 submitted or closed.

The mode is chosen by the body: `marksheetStatus` first, then `eligibleStudents`, otherwise a blueprint save (needs both `caComponents` and `finalBlueprint`). Every mode: 400 invalid JSON or missing `moduleCode`, 404 unknown module.

REFERENCE

Lecturer desk: marks and approval

/api/lecturer/marks · First Examiner

Method	Endpoint	Request	Success response	Rules and errors
GET	**/api/lecturer/marks** Load a module's marksheet	query `moduleCode`	`200 [ { id, moduleCode, studentIndex, caQuestionsMarks, finalExamQuestionsMarks, secondExamMarks, isAbsentCa, isAbsentTheory, isAbsentPractical, isAbsentFinal, isEligible } ]`	400 moduleCode missing. 404 unknown module. Sorted by student index; final marks normalised to the current blueprint. Includes `secondExamMarks`: the UI decides when to show them.
POST	**/api/lecturer/marks** Save the First Examiner's marks	`{ moduleCode, students[], deletedIndexes? }`	`200 { success, saved, created, updated, deleted, skippedDeletions, rejectedRows, stats }`	400 empty payload. 404 module. 403 frozen. 409 submitted or closed. Every value is clamped to its blueprint maximum and unknown questions are dropped. Rows that have Second Examiner marks are never deleted (`skippedDeletions`). **Never writes** `secondExamMarks`. First save moves DRAFT to MARKING. One transaction.
POST	**/api/lecturer/marks/approve** Approve after a variance	`{ moduleCode }`	`200 { success, lecturerApprovedAt, stats }` or `{ success, unchanged: true }`	400 moduleCode missing. 404 module. 409 module is not in RECONCILIATION_NEEDED. 403 frozen. Sets `lecturerApproved`, which unlocks the Second Examiner's reconciliation step.

REFERENCE

Second Examiner console

/api/examiner · 3 operations

Method	Endpoint	Request	Success response	Rules and errors
GET	**/api/examiner/marks** Modules assigned as Second Examiner	query `email` (no `moduleCode`)	`200 [ { id, code, name, credits, isFrozen, stats, assignedActiveLec } ]`	Filtered by the Second Examiner's e-mail. Without `email`, every module is returned.
GET	**/api/examiner/marks** Second-marking ledger	query `moduleCode`	`200 [ { id, moduleCode, studentIndex, finalExamQuestionsMarks, secondExamMarks, isAbsentTheory, isAbsentPractical, isAbsentFinal, isEligible } ]`	404 unknown module. `secondExamMarks` is pre-filled from the First Examiner's marks. **CA marks are deliberately not selected or returned.**
PATCH	**/api/examiner/marks** Save second marks, optionally finalise	`{ moduleCode, students: [ { studentIndex, secondExamMarks, isAbsentTheory, isAbsentPractical } ], finalize? }`	`200 { success }` with finalize: `200 { success, requiresReconciliation, newStatus }`	400 no students, no enabled final paper, or incomplete second marking (`issues[]`; any N answers per section). 404 module. 409 not handed over, not in SECOND_CHECKING, or frozen. Writes only `secondExamMarks` and absence flags. Weighted variance above 1 gives RECONCILIATION_NEEDED, otherwise FINALIZED.
POST	**/api/examiner/marks/finalise-reconciled** Close a reconciled marksheet	`{ moduleCode }`	`200 { success, reconciledAt }`	400 moduleCode missing. 404 module. 409 not in RECONCILIATION_NEEDED. 403 the First Examiner has not approved yet. The freeze flag is not checked.

RULES

State and lock matrix

What each call does for every marksheet status, and when the HOD has frozen the module

Call	Frozen by HOD	DRAFT	MARKING	SECOND CHECKING	FINALIZED	RECONCILIATION NEEDED	RECONCILED
Save marks `POST /lecturer/marks`	403	OK (a)	OK	409	409	409	409
Save blueprint `PATCH /lecturer/modules, mode B`	403	OK	OK	409	409	409	409
Set cohort size `PATCH /lecturer/modules, mode C`	403	OK	OK	409	409	409	409
Submit for second checking `PATCH /lecturer/modules, mode A`	403	OK (b)	OK (b)	no-op	409	409	409
Recall submission `PATCH /lecturer/modules, mode A`	403	OK	no-op	OK	409	409	409
Save second marks / finalise `PATCH /examiner/marks`	409	409	409	OK (c)	409	409	409
Approve after variance `POST /lecturer/marks/approve`	403	409	409	409	409	OK	409
Close reconciliation `POST /examiner/marks/finalise-reconciled`	OK (d)	409	409	409	409	OK (d)	409
Assign Second Examiner `PATCH /admin/modules`	409	OK	OK	409	409	409	409
HOD edit, freeze, delete, re-assign `PATCH and DELETE /hod/modules, PATCH /hod/lecturers`	No freeze or marksheet-status checks: allowed in every state.						

403 blocked by the HOD freeze, or by a missing approval. **409** conflicts with the marksheet state. **no-op** returns 200 with `unchanged: true`.

The Frozen column shows the result when the module is frozen and the status would otherwise allow the call.

(a) DRAFT moves to MARKING automatically. (b) Needs a defined blueprint and a complete marksheet, otherwise 400. (c) Finalising also needs complete second marking, otherwise 400. (d) Needs `lecturerApproved`, otherwise 403. The freeze flag is not checked.

Tables and JSON shapes

What the API reads and writes (Prisma schema, three tables)

system_users	accounts and roles
id	integer, primary key
email	unique, stored lower-case
role	LECTURER (default) or ADMIN
full_name	display name
password_hash	scrypt$salt$hash (legacy accounts: plain text)
is_hod, is_active_lec, is_exam_lec	capability flags, set by the Admin
recovery_email	where reset links are sent
reset_token, reset_token_expiry	one-time token, valid 1 hour
created_at	timestamp

modules	one row per module
code	unique; the key used across the API
name, credits	text; integer, default 3
eligible_students	cohort size authorised by the HOD
deadline	optional timestamp
is_frozen	HOD freeze flag
active_lecturer_id	First Examiner, link to system_users
exam_lecturer_id	Second Examiner, link to system_users
stats (JSON)	blueprint and workflow state, keys below

stats keys: moduleComponents, caComponents[], finalBlueprint, examTemplate[], secondChecking, marksheetStatus, marksheetStartedAt, submittedAt, submittedTo, recalledAt, recallCount, blueprintUpdatedAt, varianceThreshold, reconciliationRequestedAt, lecturerApproved, lecturerApprovedAt, reconciledAt, caCompletionRate

student_marks	one row per student per module
module_code, student_index	unique together; rows are deleted with their module
ca_questions_marks	{ compId: { Q1: 7, Q2: 9 } }
final_exam_questions_marks	{ theory: { mcq: {}, essay: {} }, practical: {} } (First Examiner)
second_exam_marks	same shape, written only by the Second Examiner
is_absent_ca	{ compId: true }
is_absent_theory, is_absent_practical	absence per parent section
is_absent_final	true only when both sections are absent
is_eligible	cohort eligibility, default true

modules.active_lecturer_id and exam_lecturer_id -> system_users.id (set to null when the user is deleted) | student_marks.module_code -> modules.code (cascade delete) | unique (module_code, student_index)

Responses and errors

How every route behaves, so a caller can rely on it

Transport	JSON over HTTP. 15 route handlers under `src/app/api/**/route.ts` (Next.js App Router). No version prefix.
Errors	`{ "error": "message" }`. Some add `issues[]` (completeness failures) or `marksheetStatus` (state conflicts).
Success	Mutations return `{ success: true, ... }`. Create calls return 201.
No-ops	Status already set, or approval already given: 200 with `unchanged: true`.
Identifiers	Modules by `code`. Users by numeric `id` (Admin, HOD) or `email` (auth, lecturer and examiner lists). Students by `studentIndex`, trimmed and upper-cased.
Lists	Plain arrays. No pagination, filtering or sort parameters; each route sorts server-side.
Validation	Hand-written checks; zod only for sign-up. Marks are sanitised against the blueprint on save: clamped, unknown keys dropped.
Caching	11 routes export `dynamic = "force-dynamic"`; the other 4 are POST-only auth routes.

STATUS CODES IN USE

Code	Meaning	Used for
200	**OK**	Reads, updates, and idempotent no-ops
201	**Created**	Sign-up, create user, create module
400	**Bad request**	Missing or invalid field; failed rule (weights, completeness, same examiner)
401	**Unauthorized**	Login failure only: unknown e-mail or wrong password
403	**Forbidden**	HOD freeze, missing First Examiner approval, wrong current password
404	**Not found**	Unknown user or module
409	**Conflict**	Duplicate e-mail or code, wrong marksheet state, locked assignment
500	**Server error**	Unexpected failure; the message text varies by route

NOTES

Known limitations and recommended next steps

Findings come from reading the route handlers and helpers; no penetration test was run

#	Finding	What the code does	Recommended next step
1	**No server-side authentication or authorization**	No middleware and no session or token check in any route. Role and capability guards run only in the browser (`RoleGuard`, a `localStorage` session), so any HTTP client that can reach the server can call every endpoint, including admin and delete operations. Login returns the user object but no token.	Issue a server-verified session (signed, httpOnly cookie) at login and enforce role checks in every route.
2	**Caller identity comes from the request**	Profile, change-password and the lecturer and examiner module lists identify the user by an e-mail in the query or body, not by a verified session.	Derive the user from the session and drop the e-mail parameters.
3	**Password reset depends on setup**	The link is e-mailed to the recovery e-mail only (silently skipped when none is saved). The sender is Resend's test address `onboarding@resend.dev`, and the link base comes from `NEXT_PUBLIC_APP_URL` (default localhost).	Verify a sending domain, set the app URL in production, and prompt users to save a recovery e-mail.
4	**Destructive actions have few guards**	Deleting a module cascades to all its marks with no freeze or status check. HOD re-assignment (`PATCH /api/hod/lecturers`, and the ids-only mode of `PATCH /api/hod/modules`) skips lock and capability validation. Admin create-user accepts any role string.	Reuse the lock checks, role list and capability validation already used by the Admin assignment route.
5	**Hardening gaps**	Several 500 responses echo internal error text (profile PATCH `detail`, marks and blueprint routes). The database pool disables certificate verification (`rejectUnauthorized: false`). Login has no rate limiting.	Return generic errors and log details server-side, pin the database CA certificate, add login rate limiting.
6	**Legacy plain-text passwords**	Accounts created before hashing still hold plain-text passwords. They are upgraded only when the user next changes or resets the password.	Re-hash on the next successful login, or force a reset.